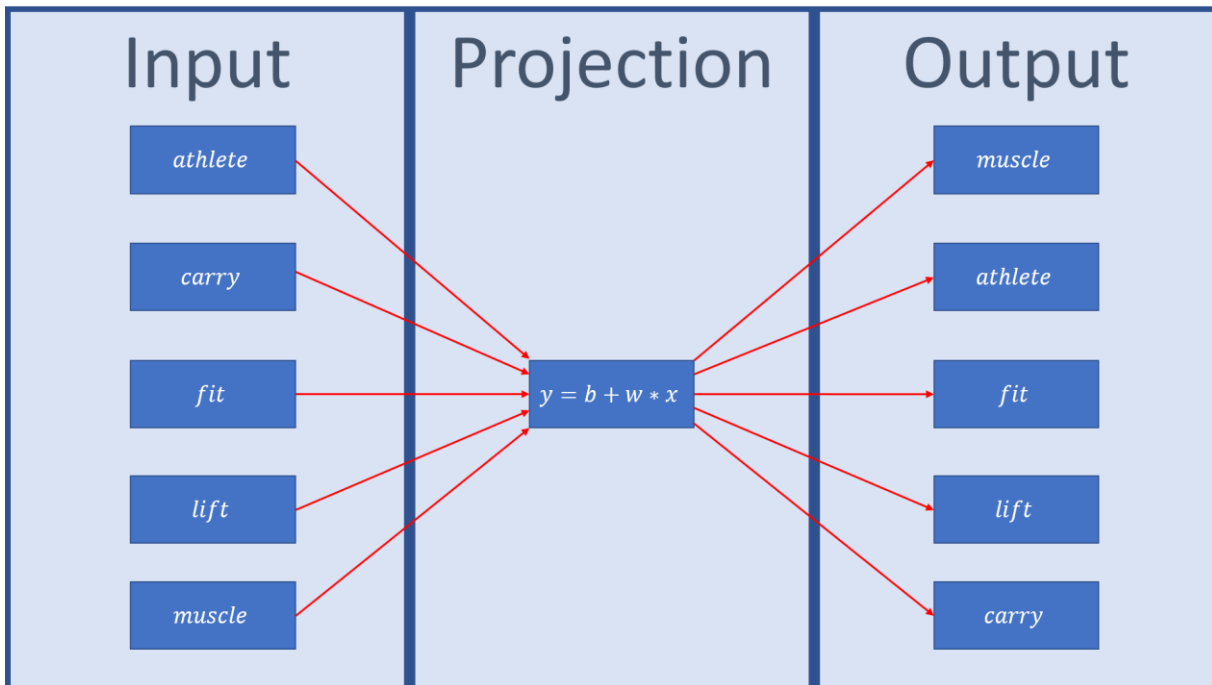
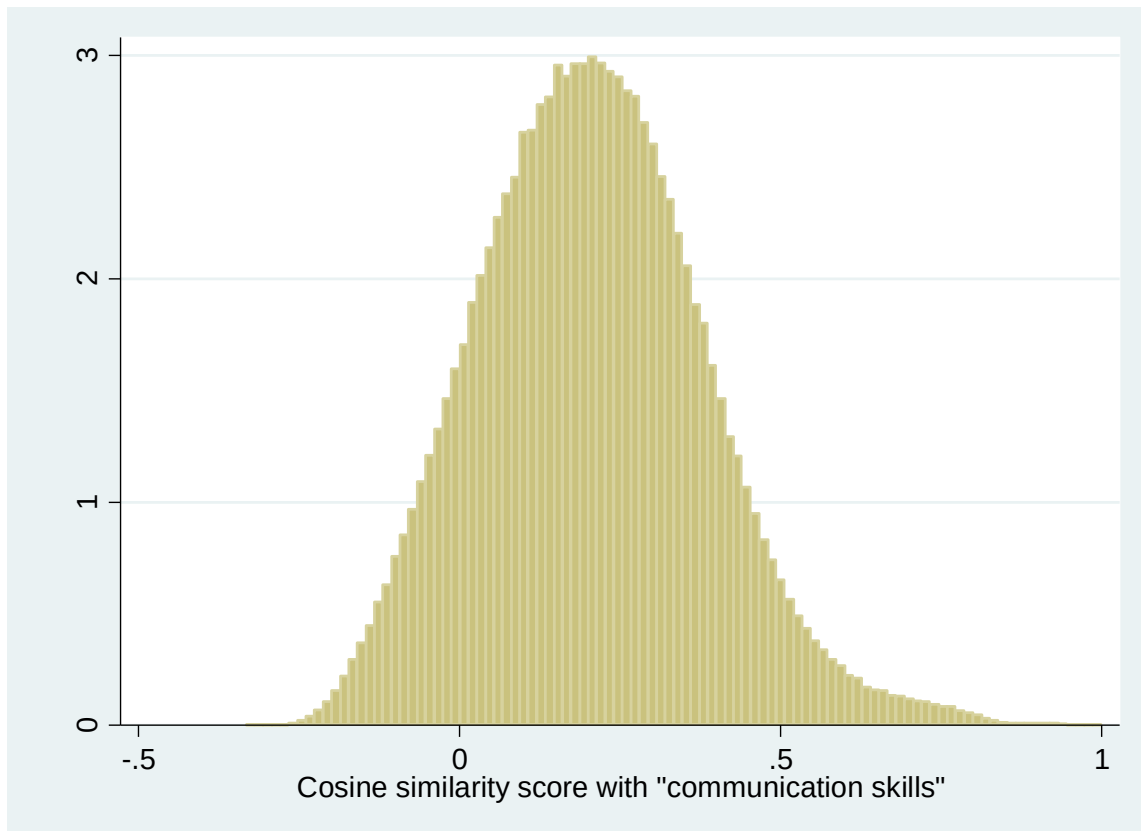


Appendix Figure A1: Visual Representation of a Hypothetical Word2Vec Neural Network

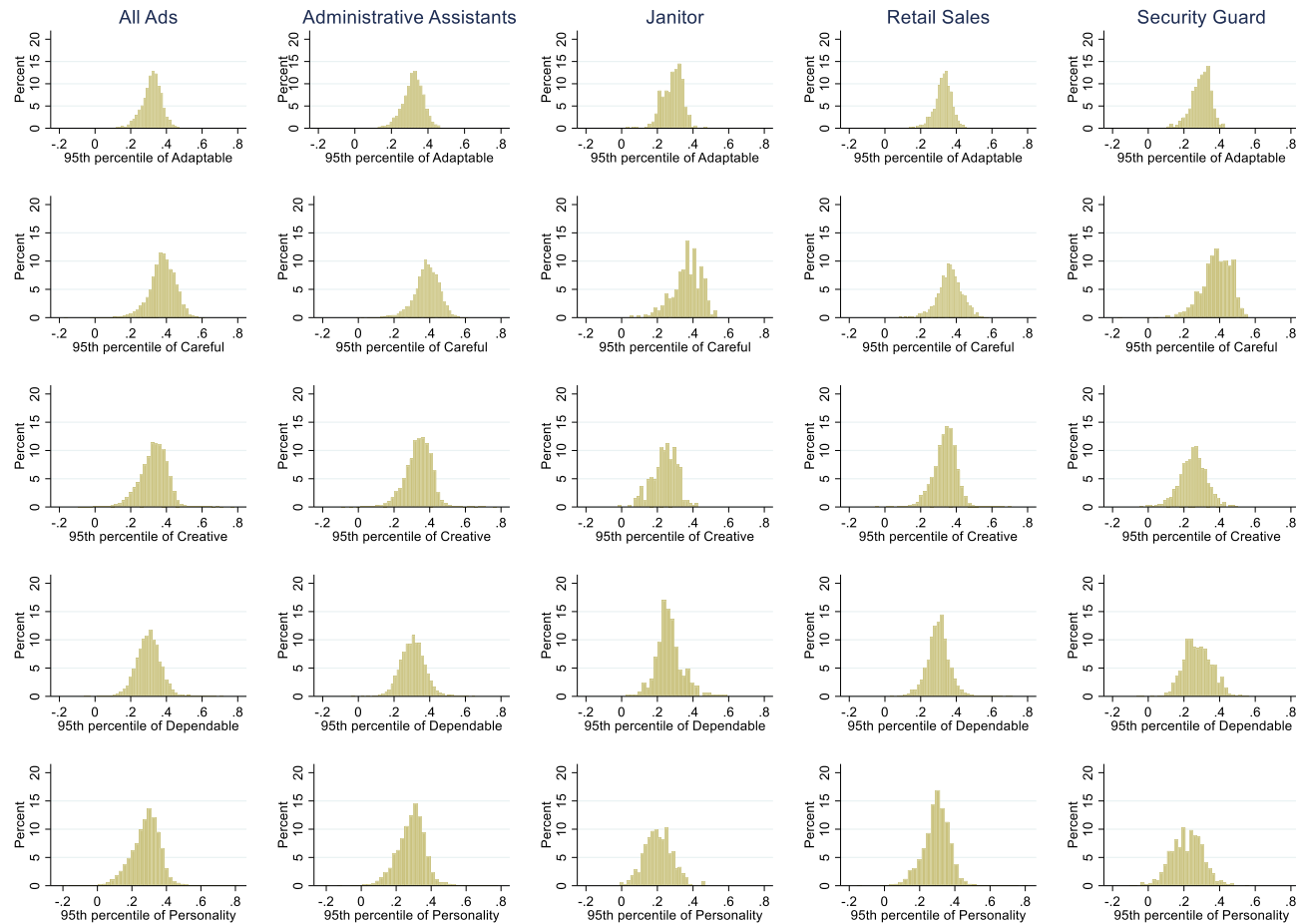


Appendix Figure A2: Example of the Distribution of Cosine Similarity (CS) Scores



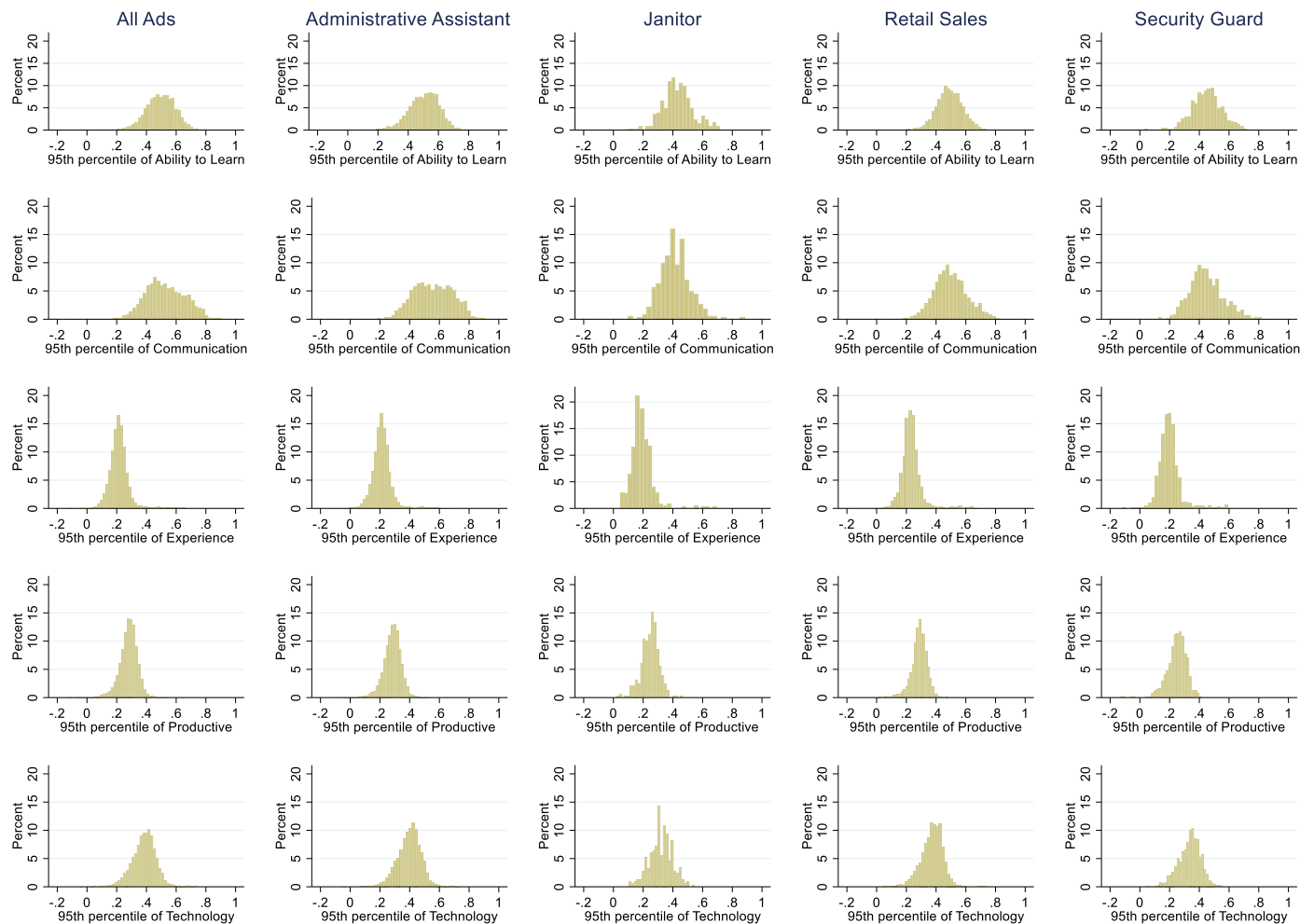
Note: Figure reports the distribution of CS scores for all trigrams from the job ads with the communication skills stereotype. The higher the CS score, the more related the trigram is to “communication skills.”

Appendix Figure A3: Distributions of 95th Percentiles of CS Scores for Stereotypes Related to Personality



Note: Data come from the job ads collected in NBB. Each panel plots the distribution of CS scores at the 95th percentile for the job ads with each stereotype related to personality. The first column contains the distribution of all the ads in our sample. The remaining columns disaggregate the job ads by occupation.

Appendix Figure A4: Distributions of 95th Percentiles of CS Scores for Stereotypes Related to Skills



Note: Data come from the job ads collected in NBB. Each panel plots the distribution of CS score at the 95th percentile for the job ads with each stereotype related to skills. The first column contains the distribution of all the ads in our sample. The remaining columns disaggregate the job ads by occupation.

Appendix Table A1: Text of Trigrams at the 95th Percentile of CS Scores within Job Ads

		Trigrams closest to 1 standard deviation above mean				
	Mean 95th percentile	2 trigrams below	1 trigram below	Exactly 1 standard deviation above the mean	1 trigram above	2 trigrams above
	(1)	(2)	(3)	(4)	(5)	(6)
Attractive	friendly articulate personality	position seek energetic	interpersonal skills valued	passion fashion luxury	ships museum looking	cheerful ability multitask
Hearing	benefits package consideration	bridges eye care	paper work filing	hour noncommissioned officers	regard billing procedures	community seeking evening
Memory	applicant must good	background retail knowledge	oriented ability follow	time frame attitude	directs person matter	excel spreadsheets different
Physically able	assistant position available	fast paced fun	experience preferred necessary	work preferred necessary	required flexibility required	anything required help
Adaptable	making learning agility	must reliable apply	duties needed excellent	dedicated providing well	individual fast learner	moment respectful willing
Careful	computer skills required	basis important able	utmost professionalism integrity	processing payments must	requirements need really	assistant good verbal
Creative	position energetic detail	support vp marketing	office experience ability	involves spearheading independent	professional personal presentation	experience skills essential
Dependable	media reputation management	reliable transportation needed	service skills critical	hours retirees welcome	professional attitude well	experience excellent telephone
Personality	women feel con	correct visual image	attention detail outstanding	outgoing highly motivated	associate tribeca passion	luxury experience previous
Ability to learn	word excel must	sales skills managing	communication skills competency	career using cashiering	excellent people communication	abilities ability complete
Communication skills	politeness absolutely fundamental	backend customers required	skills abilities ability	client relation skills	filing typing computer	communication organizational skills
Experienced	evenings weekends requires	computer knowledge exciting	duties qualifications experience	level gain exposure	quickly become one	full time receptionist
Productive	fun eventful one	banker seeking ambitious	computer literate interested	tasks experience necessary	organization efficiency must	player good communication
Technology	paced integrative medical	ability learn new	willingness learn new	social media google	communication skills proficiency	computer skills proficiency

Note: For each stereotype, we construct the distribution of trigrams at the 95th percentile of all trigrams in a job ad. We identify the trigram that is closest to the mean 95th percentile trigram and the trigram that is closest to one standard deviation above the median. We also show the two trigrams immediately below 1 standard deviation above the mean 95th percentile (columns 2 and 3) and the two trigrams immediately above 1 standard deviation above the mean 95th percentile (columns 5 and 6). The CS scores in columns (2), (3), (5), and (6) were very close to those in column (4), for each row. For example, in the first row (Attractive), the CS scores in columns (2)-(6) ranged from 0.3772 to 0.3776. The CS score in column (1) was 0.3082.

Appendix Table A2: Analysis of Discrimination Against Younger Applicants

		Female		Male		
		Admin	Sales	Janitor	Sales	Security
		(1)	(2)	(3)	(4)	(5)
<i>Baseline discrimination</i>		0.048	0.074	0.115	0.077	0.090
<i>Health stereotypes</i>	<i>Predicted sign</i>					
Attractive	Negative	0.004 (0.003)	-0.000 (0.010)	0.007 (0.022)	-0.010 (0.009)	-0.000 (0.012)
Hearing	Negative	-0.002 (0.002)	-0.009 (0.007)	0.036* (0.015)	-0.005 (0.006)	-0.005 (0.008)
Memory	Negative	-0.001 (0.003)	0.011 (0.008)	0.024 (0.020)	0.001 (0.008)	0.009 (0.009)
Physical Ability	Negative	-0.002 (0.003)	0.003 (0.009)	0.025 (0.022)	-0.006 (0.009)	0.001 (0.009)
<i>Personality stereotypes</i>	<i>Predicted sign</i>					
Adaptable	Negative	-0.003 (0.003)	0.028** (0.010)	0.021 (0.026)	0.008 (0.009)	-0.002 (0.011)
Careful	Positive	0.005 (0.003)	-0.022* (0.009)	-0.026 (0.022)	-0.004 (0.009)	0.002 (0.010)
Creative	Negative	-0.003 (0.003)	0.016 (0.008)	-0.020 (0.023)	-0.015 (0.008)	0.010 (0.012)
Dependable	Positive	-0.006* (0.003)	-0.001 (0.008)	0.003 (0.018)	-0.013 (0.008)	0.010 (0.009)
Personality	Positive/Negative	-0.002 (0.003)	-0.010 (0.008)	0.018 (0.023)	-0.001 (0.008)	0.004 (0.010)
<i>Skills stereotypes</i>	<i>Predicted sign</i>					
Ability to Learn	Negative	0.000 (0.003)	-0.007 (0.010)	-0.056* (0.027)	-0.004 (0.010)	-0.008 (0.012)
Communication Skills	Positive/Negative	0.005 (0.003)	-0.007 (0.011)	0.085* (0.037)	0.014 (0.011)	-0.023 (0.014)
Experienced	Positive	0.002 (0.002)	-0.003 (0.006)	-0.018 (0.015)	-0.008 (0.006)	0.005 (0.006)
Productive	Positive/Negative	-0.004 (0.003)	-0.003 (0.008)	-0.011 (0.025)	0.015 (0.009)	-0.018 (0.011)
Technology	Positive	0.002 (0.003)	-0.005 (0.008)	-0.035 (0.018)	-0.003 (0.007)	-0.001 (0.009)
N		14136	2834	646	3274	1804
Adjusted R ²		0.035	0.040	0.082	0.042	0.093

Note: See notes to Table 4. The difference is that the outcome is now defined as discrimination against younger applicants. Note that the sample sizes do not add up because we do not observe reverse discrimination in all city-occupation cells, so those observations are excluded from the probit. * indicates that the estimate is statistically significant at the 5% level. ** indicates that the estimate is statistically significant at the 1% level.

Appendix Table A3: Comparing Words from Alternative Corpora

Stereotype	Complete Wikipedia		Spider-crawl Wikipedia		Strict string match
	Word	CS Score	Word	CS Score	Word
Attractive	unattractive	0.7458	desirable	0.6811	attractive
	appealing	0.6966	advantageous	0.6603	
	desirable	0.6878	unattractive	0.6250	
	unappealing	0.6699	appealing	0.6150	
	alluring	0.6612	hospitable	0.5845	
	enticing	0.6471	palatable	0.5710	
	ideal	0.6290	favorable	0.5578	
	advantageous	0.6268	apt	0.5564	
	agreeable	0.6207	adaptable	0.5564	
	interesting	0.6175	comfortable	0.5479	
Hearing	hearings	0.5888	judgment	0.4814	hearing
	testifying	0.5629	deaf	0.4631	
	testimony	0.5403	hearings	0.4614	
	arraignment	0.5272	tinnitus	0.4585	
	pleading	0.5148	earplugs	0.4549	
	sentencing	0.5078	testifying	0.4473	
	testified	0.5051	cochlear	0.4447	
	committal	0.4971	proceedings	0.4426	
	questioning	0.4929	auditory	0.4397	
	complaint	0.4853	trial	0.4374	
Memory	memories	0.6628	memories	0.7152	memory
	brain	0.5274	amnesia	0.5725	
	recollection	0.5185	hippocampus	0.5631	
	cpu	0.5138	cognition	0.5625	
	remembering	0.5051	cognitive	0.5617	
	eidetic	0.4975	retrieval	0.5589	
	scratchpad	0.4933	recollection	0.5514	
	cache	0.4873	episodic	0.5478	
	rom	0.4862	perceptual	0.5418	
	consciousness	0.4770	reconsolidation	0.5093	
Physically able	unable	0.6887	unable	0.6910	physically able
	enough	0.6050	willing	0.5993	
	willing	0.5912	enough	0.5800	
	trying	0.5904	psychologically	0.5680	
	ability	0.5865	trying	0.5513	
	needed	0.5696	unwilling	0.5365	
	attempting	0.5556	expected	0.5363	
	allowed	0.5549	anxious	0.5294	
	psychologically	0.5535	attempting	0.5281	
	unwilling	0.5505	eager	0.5271	
Number of articles		appx. 5,500,000	65,532		
Number of words		885,424	260,073		

Note: The table presents examples of result from training models on different corpora. Complete Wikipedia includes all the articles; it has 885,424 words and 200-dimensional vectors. Spider-crawl (scrapy spider) Wikipedia starts with articles referring to stereotypes, ageism, and labor markets, as explained in the text. Both use 200-dimensional vectors. The “Word” column lists the words with the highest similarity scores; the scores are reported in the “CS Score” column.

Online Appendix B: Supplemental Analyses

Construction of Phrases

One robustness analysis we performed was to alter how many words we use in job-ad phrases. As our baseline, we used trigrams. We made this choice prior to analyzing the relationships between the phrases, stereotypes, and measured discrimination, to avoid the risk of cherry-picking phrase length to get stronger results. Nonetheless, we can use our same methods to construct similarity scores for phrases of any length.

Thus, subsequent to our baseline analysis, we redid our analysis using two-word and four-word phrases. The results, shown in Appendix Figures B1A and B1B, provide evidence that the results we obtained using three-word phrases are quite robust to using two- or four-word phrases.⁶² The signs of the estimates are similar using different phrase lengths. For women, varying the length of phrases does not result in more significant estimates, indicating that the lack of strong associations between stereotyped language and discrimination against older women is not an artifact of phrase length. There is some heterogeneity in the statistical significance of the estimates, even though the estimates are similar in sign and magnitude. For men, the results from Figure 6 are very robust to changing the phrase length. Many of the stereotypes we found to predict discrimination using trigrams have similar estimated coefficients for predicting discrimination using either bigrams or quadgrams, although the statistical significance varies (less so for men than for women). Appendix Figure B1B suggests that the results are perhaps more robust when comparing trigrams to quadgrams and less robust when comparing bigrams to trigrams.

This may be because two-word phrases less reliably reflect the underlying stereotype – as evidence by the mean 95th percentile CS scores being lower for bigrams.⁶³

⁶² The regression results are reported in Appendix Tables B1A and B1B.

⁶³ In addition to this supplemental analysis, in Appendix Tables B2A and B2B and Appendix Figure B2, we replicate our baseline results using the median and the maximum CS scores rather than the 95th percentile. The results are consistent with the concerns we discussed earlier about using the median or maximum. In a number of age-occupation-cells, we find different stereotypes predicting discrimination at different percentiles. This variation is driven by two factors. First, the estimates (regardless of significance) are often very different at the median than they are in the upper tail, flipping signs when we move from the median to the 95th percentile, but much more rarely comparing results using the 95th percentile vs. the maximum. The implication is that more of the results at the median conflict with expectations based on the industrial psychology literature. This is not surprising for the median, where many of the phrases are unrelated to the stereotype. There are also some differences between the 95th percentile and the maximum, although rarely in terms of sign for the larger coefficient estimates. Looking back at Figure 4, this is not surprising, since the shapes of the distributions are sometimes very different for the maximum than for the 95th percentile.

Coding of Experience

We also explore modifying the method of characterizing age stereotypes in ads when an ad seeks more or less of a stereotypical characteristic. The semantic similarity model struggles when the text of a job ad features adverbs modifying the stereotyped characteristic. This seems most salient for the stereotype that older workers have more experience. Job ads that state they want a worker with ten years of experience will use language that is just as related to experience as job ads that state that they only require one to two years of experience or that no experience is required. Our results for experience are then simply the average direction of the relationship. However, because language associated with experience may positively affect the hiring of older workers and language associated with low levels of experience or no experience may negatively affect the hiring of older workers, our results are potentially biased towards zero. We therefore tried to modify the coding of experience to determine the impact of this potential problem.

In particular, we identified every sentence in the job ads with the word “experience.” We then searched for preceding or following words (10 words before or after) that indicated a context other than work experience (such as customer experience or shopping experience). We also capture instances of “commensurate” and “depending,” which typically appear in the text to indicate that pay is based on experience. We coded these instances as not requiring experience. We then identified all instances where the words “no” or “not” were used immediately before or after experience, which allowed us to capture phrases such as “no experience was necessary” or “experience not needed,” and coded these instances as not requiring experience. To code low experience requirements, we search ten words before or after the use of experience for any instance of “year/years.” If “year” or “years” is preceded by the number 1 or 2 and there are no numbers higher than 2, we code it as low experience required. No or low experience could, of course, indicate a preference for a younger worker.

These kinds of refinements of the semantic coding will always suffer from false positives and false negatives because the meanings of words are subtle, and combinations of words can give rise to different meanings, which algorithmic coding cannot definitively capture. After many rounds of refinements to arrive at the recoding described above, we found that our algorithm had a false positive rate of 24% for coding “no

experience” and a false positive rate of 33% for coding “low experience.”⁶⁴ These false positive rates reflect the difficulty of designing this kind of algorithm to capture semantic meaning based on specific words or phrases. This difficulty, even in this reasonably simple context in which one might want to “override” the machine learning, demonstrated the value of the machine learning – even putting aside the clear advantages of machine learning for protecting from subjectivity and increasing replicability and transparency.⁶⁵

To test how controlling for no or low experience requirements in a job ad impacts the estimates, we interact dummy variables for the job ad having no experience requirement (N_j) or a low experience requirement (L_j) with the cosine similarity score for experience (P_{jExp}^{95}). We continue to control for the other thirteen stereotypes, so our model becomes:

$$(B1) \quad Pr[d_{exp}^{ij} = 1] = \alpha + \sum_s \beta_s P_{js}^{95} + \gamma_1 N_j + \theta_1 (P_{jExp}^{95} \times N_j) + \gamma_2 L_j + \theta_2 (P_{jExp}^{95} \times L_j) + X_{ij} \delta + \varepsilon_{ij} .$$

Appendix Table B3A reports the results for women. In the baseline estimates, we found no significant relationship between stereotyped language related to experience and discrimination. We find no evidence that job ad language related to no or low experience is associated with discrimination, nor do we find evidence of interactions of indicators for low or no experience with the CS score with experience. Appendix Table B3B reports the results for men. In this case, our refinement produces more nuanced results in a handful of cases, but for many age-occupation clusters the results are the same. In the baseline estimates, we find no significant correlation between stereotyped language related to experience and discrimination, except for janitors (both middle-aged and old). When we introduce the controls and interactions for job ads using language indicating no or low experience required, we find significant increases in discrimination when the ad uses phrases that indicate low experience in retail sales for middle-aged men. Ads with this language increase the rate of discrimination by 19.5 percentage points. We find no evidence that language indicating no experience required is correlated with discrimination in any age-occupation cluster (for men or women).

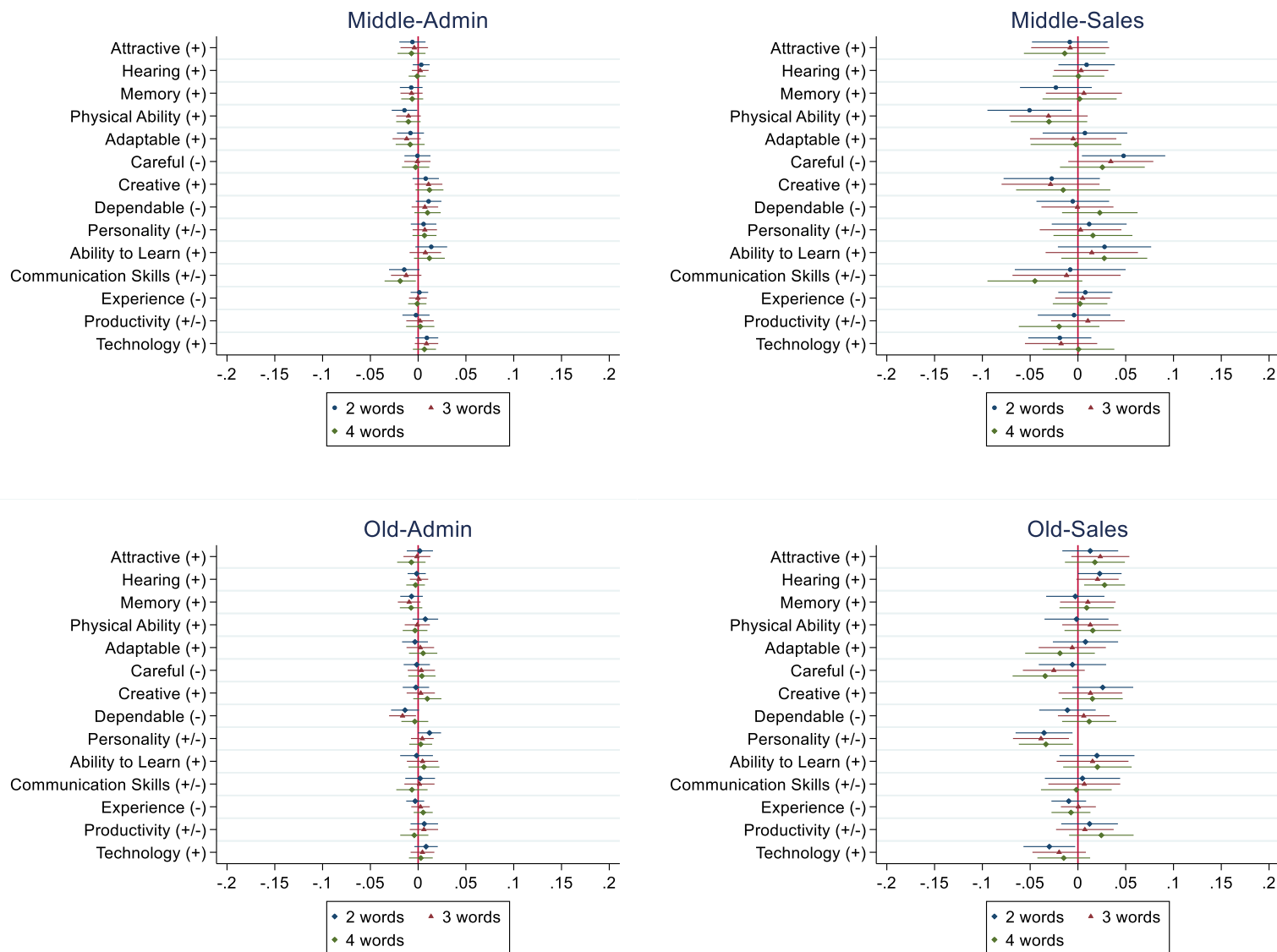
⁶⁴ Our subjective coding of a random sample of ads, and comparisons to the algorithmic coding, are available upon request.

⁶⁵ Ningrum et al. (2020) explore a similar approach to classifying job ads by whether they include references to specific groups (e.g., gender or age), fine tuning how the language is read to better capture how these group identifiers are referenced. We strongly suspect that this approach would be far less suitable to characterizing the relationship between job-ad language and stereotypes, since stereotypes can be expressed in many ways, making it more important to measure linguistic similarity even when words do not match.

When we allow the language related to experience to have different effects on discrimination when the job ad features language related to no or low experience, we find significant differences for middle-aged janitors and older men in retail sales. In both cases, the rates of discrimination are lower on ads that feature language highly related to experience and use phrases indicating they require only one to two years of experience, compared to the omitted group of language indicating that more experience is required. These somewhat non-intuitive results lead us to conclude that this exercise may not be reliable at providing better evidence on the effects of job ad language related to experience.⁶⁶

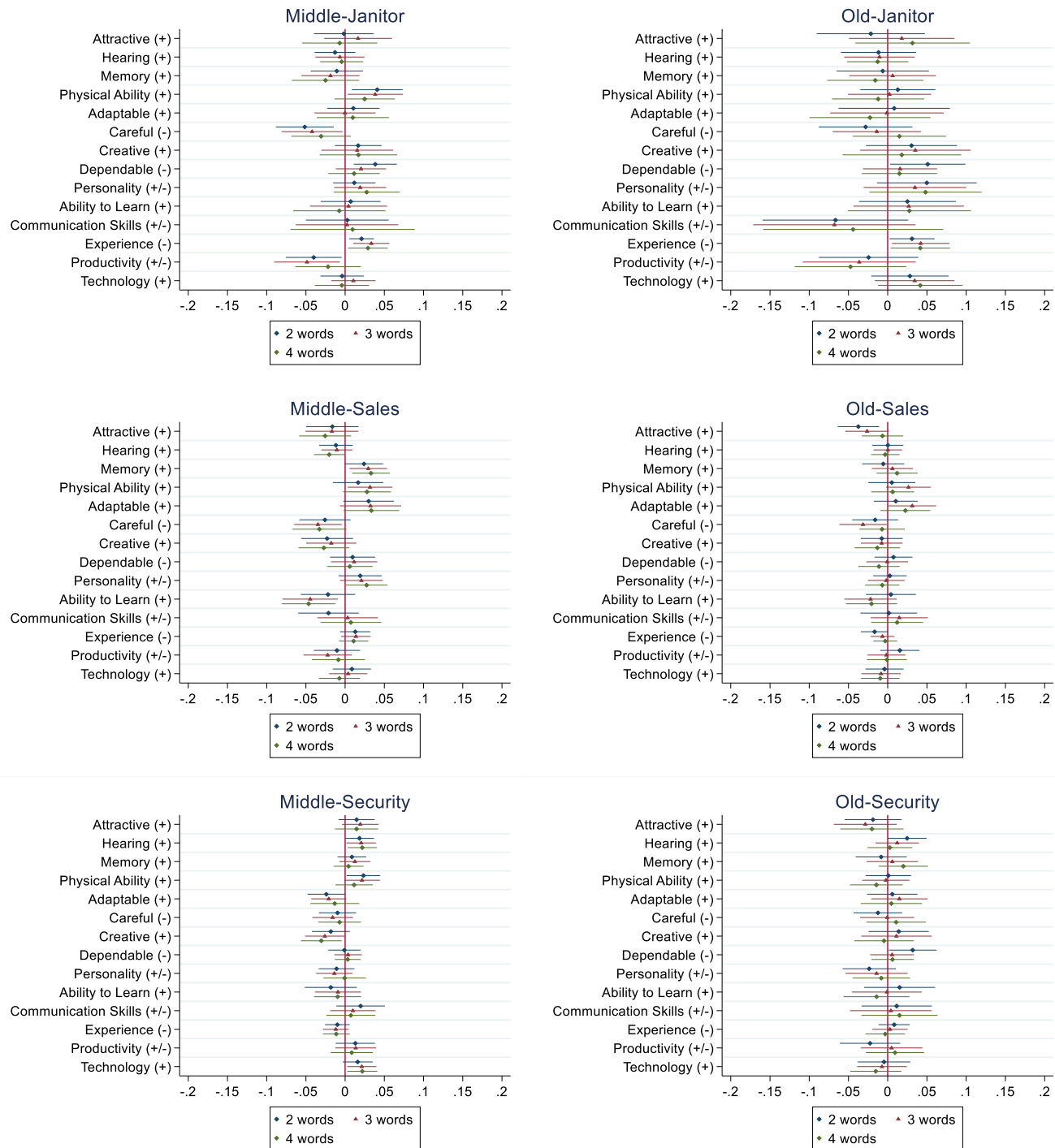
⁶⁶ It is also the case, though, that at the end of the day this exercise may not matter too much, because our coding (based on a random sample) indicated that nearly three-quarters of job ads with language related to experience had the meaning of more experience required or preferred.

Appendix Figure B1A: Varying the Number of Words in Phrases, Women



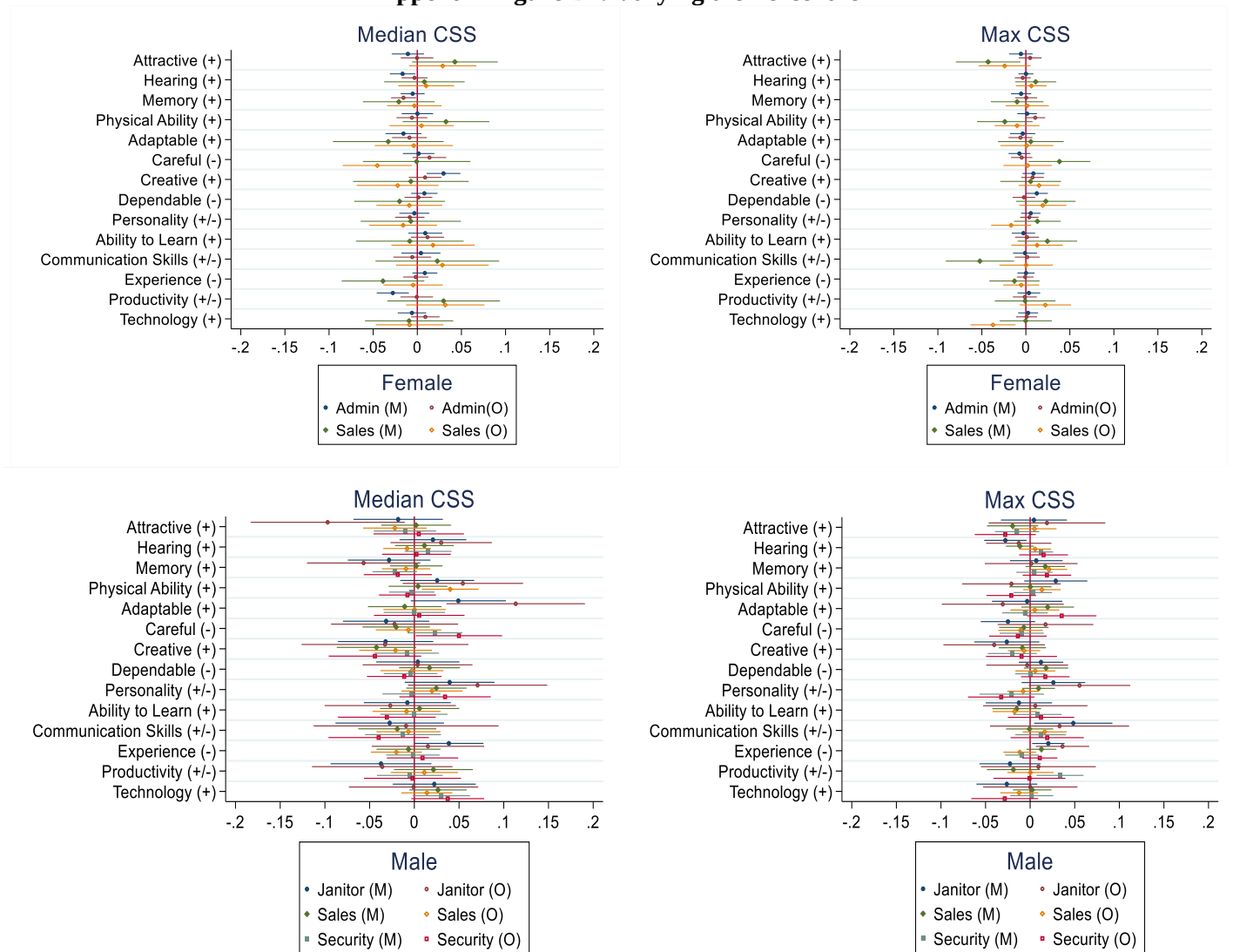
Note: See notes to Figure 6. The difference is that here we vary the number of words in a phrase used to calculate the cosine similarity scores.

Appendix Figure B1B: Varying the Number of Words in Phrases, Men



Note: See notes to Figure 6. The difference is that here we vary the number of words in a phrase used to calculate the cosine similarity scores.

Appendix Figure B2: Varying the Percentile



Note: Figure reports estimated marginal effects from equation (10) and 95% confidence intervals, using the median CS score and the maximum CS score. Positive predictions indicate we expect to see more discrimination against older applicants, negative predictions indicate we expect to see less discrimination against older applicants. Standard errors are clustered at the job-ad level. The age of the sample is indicated in parentheses, M for middle-aged and O for old.

Appendix Table B1A: Varying the Number of Words in Phrases, Women, Effects on Discrimination Against Older Applicants

# words in phrase	Middle-Admin			Middle-Sales			Old-Admin			Old-Sales		
	2	3	4	2	3	4	2	3	4	2	3	4
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
<i>Health stereotypes</i>												
Attractive (+)	-0.006 (0.007)	-0.004 (0.007)	-0.007 (0.007)	-0.009 (0.020)	-0.008 (0.021)	-0.014 (0.022)	0.002 (0.007)	-0.001 (0.007)	-0.007 (0.008)	0.013 (0.015)	0.023 (0.015)	0.018 (0.016)
Hearing (+)	0.003 (0.005)	0.002 (0.004)	-0.001 (0.005)	0.009 (0.015)	0.003 (0.015)	0.001 (0.014)	-0.002 (0.005)	0.001 (0.005)	-0.003 (0.005)	0.023 (0.012)	0.021 (0.011)	0.028* (0.011)
Memory (+)	-0.007 (0.006)	-0.007 (0.006)	-0.006 (0.006)	-0.023 (0.019)	0.006 (0.020)	0.002 (0.020)	-0.007 (0.006)	-0.009 (0.006)	-0.007 (0.006)	-0.003 (0.016)	0.010 (0.015)	0.009 (0.015)
Physical Ability (+)	-0.014* (0.007)	-0.010 (0.007)	-0.010 (0.007)	-0.051* (0.022)	-0.031 (0.021)	-0.030 (0.020)	0.008 (0.007)	-0.001 (0.007)	-0.003 (0.007)	-0.001 (0.017)	0.013 (0.015)	0.016 (0.015)
<i>Personality stereotypes</i>												
Adaptable (+)	-0.008 (0.007)	-0.012 (0.008)	-0.008 (0.008)	0.007 (0.023)	-0.005 (0.023)	-0.002 (0.024)	-0.003 (0.007)	0.002 (0.007)	0.005 (0.008)	0.008 (0.017)	-0.006 (0.018)	-0.019 (0.019)
Careful (-)	-0.001 (0.007)	-0.001 (0.007)	-0.003 (0.007)	0.048* (0.022)	0.034 (0.023)	0.026 (0.023)	-0.001 (0.007)	0.003 (0.007)	0.004 (0.007)	-0.006 (0.018)	-0.025 (0.017)	-0.034* (0.017)
Creative (+)	0.008 (0.007)	0.011 (0.007)	0.012 (0.007)	-0.027 (0.026)	-0.029 (0.026)	-0.015 (0.025)	-0.002 (0.007)	0.003 (0.008)	0.010 (0.008)	0.026 (0.016)	0.013 (0.017)	0.015 (0.016)
Dependable (-)	0.011 (0.007)	0.007 (0.007)	0.010 (0.007)	-0.005 (0.019)	-0.001 (0.019)	0.023 (0.020)	-0.014 (0.007)	-0.016* (0.007)	-0.003 (0.007)	-0.011 (0.015)	0.006 (0.014)	0.012 (0.014)
Personality (+/-)	0.006 (0.007)	0.007 (0.006)	0.007 (0.006)	0.012 (0.020)	0.003 (0.022)	0.016 (0.021)	0.012 (0.006)	0.004 (0.006)	0.003 (0.006)	-0.035* (0.015)	-0.039** (0.015)	-0.033* (0.014)
<i>Skills stereotypes</i>												
Ability to Learn (+)	0.014 (0.008)	0.008 (0.008)	0.012 (0.008)	0.028 (0.025)	0.014 (0.025)	0.028 (0.023)	-0.002 (0.009)	0.005 (0.008)	0.006 (0.008)	0.020 (0.020)	0.015 (0.019)	0.020 (0.018)
Comm Skills (+/-)	-0.015 (0.008)	-0.013 (0.008)	-0.019* (0.008)	-0.008 (0.030)	-0.012 (0.029)	-0.045 (0.025)	0.002 (0.008)	0.001 (0.008)	-0.007 (0.008)	0.005 (0.020)	0.007 (0.019)	-0.002 (0.019)
Experienced (-)	0.001 (0.005)	-0.000 (0.005)	-0.001 (0.005)	0.008 (0.014)	0.005 (0.015)	0.002 (0.015)	-0.003 (0.005)	0.002 (0.005)	0.005 (0.005)	-0.010 (0.009)	0.000 (0.009)	-0.007 (0.010)
Productive (+/-)	-0.002 (0.007)	0.002 (0.007)	0.002 (0.008)	-0.004 (0.019)	0.010 (0.020)	-0.020 (0.022)	0.006 (0.007)	0.006 (0.008)	-0.004 (0.008)	0.012 (0.015)	0.007 (0.015)	0.024 (0.017)
Technology (+)	0.009 (0.006)	0.009 (0.006)	0.007 (0.006)	-0.019 (0.017)	-0.018 (0.019)	0.001 (0.019)	0.008 (0.006)	0.005 (0.006)	0.003 (0.006)	-0.030* (0.014)	-0.020 (0.014)	-0.015 (0.014)
N	6,822	6,822	6,821	986	986	986	7,321	7,321	7,320	1,856	1,856	1,856

Note: See notes to Table 4. * indicates that the estimate is statistically significant at the 5% level. ** indicates that the estimate is statistically significant at the 1% level.

Appendix Table B1B: Varying the Number of Words in Phrases, Men, Effects on Discrimination Against Older Applicants

	Middle-Janitor			Middle-Sales			Middle-Security			Old-Janitor			Old-Sales			Old-Security		
# words in phrase	2	3	4	2	3	4	2	3	4	2	3	4	2	3	4	2	3	4
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)	(15)	(16)	(17)	(18)
<i>Health stereotypes</i>																		
Attractive (+)	-0.002 (0.019)	0.017 (0.022)	-0.007 (0.024)	-0.016 (0.017)	-0.017 (0.017)	-0.025 (0.017)	0.015 (0.012)	0.019 (0.012)	0.015 (0.014)	-0.021 (0.035)	0.018 (0.034)	0.032 (0.037)	-0.037** (0.013)	-0.026 (0.014)	-0.007 (0.013)	-0.019 (0.018)	-0.029 (0.020)	-0.020 (0.021)
Hearing (+)	-0.013 (0.013)	-0.007 (0.016)	-0.004 (0.014)	-0.012 (0.011)	-0.010 (0.010)	-0.020* (0.010)	0.018* (0.009)	0.020* (0.010)	0.022* (0.009)	-0.012 (0.024)	-0.010 (0.023)	-0.013 (0.020)	0.000 (0.010)	0.000 (0.009)	-0.003 (0.009)	0.025* (0.013)	0.012 (0.014)	0.003 (0.015)
Memory (+)	-0.011 (0.017)	-0.019 (0.019)	-0.025 (0.022)	0.024 (0.012)	0.030* (0.012)	0.033** (0.012)	0.009 (0.009)	0.012 (0.010)	0.004 (0.010)	-0.006 (0.030)	0.006 (0.028)	-0.016 (0.031)	-0.006 (0.014)	0.006 (0.013)	0.012 (0.013)	-0.008 (0.017)	0.006 (0.017)	0.020 (0.016)
Physical Ability (+)	0.041* (0.017)	0.038* (0.018)	0.025 (0.020)	0.017 (0.016)	0.032* (0.015)	0.028 (0.016)	0.023* (0.011)	0.022 (0.012)	0.011 (0.012)	0.013 (0.024)	0.002 (0.027)	-0.012 (0.030)	0.005 (0.015)	0.026 (0.015)	0.006 (0.014)	0.001 (0.015)	-0.002 (0.015)	-0.014 (0.017)
<i>Personality stereotypes</i>																		
Adaptable (+)	0.011 (0.017)	-0.000 (0.020)	0.010 (0.023)	0.030 (0.016)	0.032 (0.020)	0.033 (0.018)	-0.024 (0.012)	-0.021 (0.011)	-0.013 (0.016)	0.008 (0.036)	-0.001 (0.037)	-0.023 (0.039)	0.010 (0.014)	0.031* (0.016)	0.023 (0.016)	0.006 (0.016)	0.015 (0.018)	0.005 (0.020)
Careful (-)	-0.051** (0.019)	-0.042* (0.020)	-0.031 (0.019)	-0.026 (0.017)	-0.035* (0.015)	-0.033 (0.018)	-0.010 (0.012)	-0.016 (0.013)	-0.007 (0.014)	-0.028 (0.030)	-0.014 (0.029)	0.015 (0.030)	-0.016 (0.015)	-0.031* (0.015)	-0.007 (0.015)	-0.012 (0.016)	-0.001 (0.018)	0.011 (0.019)
Creative (+)	0.017 (0.015)	0.015 (0.023)	0.017 (0.025)	-0.023 (0.017)	-0.018 (0.016)	-0.027 (0.017)	-0.018 (0.012)	-0.026* (0.013)	-0.030* (0.013)	0.030 (0.030)	0.035 (0.036)	0.018 (0.039)	-0.008 (0.014)	-0.008 (0.014)	-0.013 (0.015)	0.014 (0.020)	0.011 (0.023)	-0.005 (0.019)
Dependable (-)	0.038** (0.014)	0.020 (0.016)	0.011 (0.017)	0.010 (0.015)	0.011 (0.015)	0.006 (0.015)	-0.001 (0.010)	0.004 (0.009)	0.003 (0.008)	0.051* (0.025)	0.016 (0.024)	0.015 (0.024)	0.007 (0.012)	-0.001 (0.014)	-0.011 (0.013)	0.032* (0.016)	0.006 (0.014)	0.006 (0.014)
Personality (+/-)	0.012 (0.014)	0.019 (0.017)	0.027 (0.021)	0.019 (0.014)	0.021 (0.014)	0.027* (0.014)	-0.011 (0.012)	-0.014 (0.012)	-0.001 (0.014)	0.050 (0.032)	0.035 (0.033)	0.048 (0.036)	0.003 (0.011)	-0.002 (0.012)	-0.007 (0.011)	-0.024 (0.017)	-0.014 (0.020)	-0.008 (0.019)
<i>Skills stereotypes</i>																		
Ability to Learn (+)	0.007 (0.019)	0.004 (0.025)	-0.007 (0.030)	-0.022 (0.018)	-0.045* (0.018)	-0.047** (0.017)	-0.018 (0.017)	-0.009 (0.015)	-0.010 (0.015)	0.025 (0.032)	0.027 (0.036)	0.028 (0.040)	0.004 (0.016)	-0.022 (0.017)	-0.021 (0.017)	0.015 (0.023)	-0.001 (0.023)	-0.014 (0.021)
Comm Skills (+/-)	0.003 (0.027)	0.002 (0.033)	0.010 (0.040)	-0.021 (0.020)	0.003 (0.020)	0.007 (0.020)	0.020 (0.016)	0.010 (0.015)	0.007 (0.016)	-0.067 (0.047)	-0.068 (0.053)	-0.044 (0.059)	0.001 (0.018)	0.015 (0.019)	0.012 (0.017)	0.011 (0.023)	0.004 (0.027)	0.015 (0.025)
Experienced (-)	0.021** (0.008)	0.033** (0.012)	0.029* (0.013)	0.013 (0.010)	0.014 (0.010)	0.011 (0.010)	-0.010 (0.008)	-0.012 (0.008)	-0.011 (0.009)	0.031* (0.015)	0.042* (0.019)	0.042* (0.019)	-0.017 (0.009)	-0.007 (0.008)	-0.003 (0.008)	0.008 (0.010)	0.003 (0.011)	-0.003 (0.013)
Productive (+/-)	-0.040* (0.018)	-0.049* (0.021)	-0.022 (0.021)	-0.010 (0.015)	-0.022 (0.016)	-0.009 (0.017)	0.013 (0.013)	0.014 (0.013)	0.008 (0.014)	-0.024 (0.032)	-0.036 (0.037)	-0.047 (0.036)	0.015 (0.013)	-0.002 (0.012)	-0.001 (0.013)	-0.023 (0.020)	0.005 (0.020)	0.009 (0.019)
Technology (+)	-0.004 (0.014)	0.011 (0.014)	-0.004 (0.018)	0.009 (0.012)	0.004 (0.012)	-0.007 (0.013)	0.016 (0.010)	0.021* (0.009)	0.022* (0.010)	0.028 (0.025)	0.034 (0.026)	0.042 (0.027)	-0.004 (0.012)	-0.009 (0.013)	-0.009 (0.012)	-0.005 (0.017)	-0.007 (0.016)	-0.015 (0.017)
N	311	311	311	1,612	1,612	1,610	954	954	953	318	318	318	1,680	1,680	1,680	932	932	931

Note: See notes to Table 4. * indicates that the estimate is statistically significant at the 5% level. ** indicates that the estimate is statistically significant at the 1% level.

Appendix Table B2A: Varying the Percentile, Women, Effects on Discrimination Against Older Applicants

Percentile	Middle-Admin			Middle-Sales			Old-Admin			Old-Sales		
	Median	95 th	Max	Median	95 th	Max	Median	95 th	Max	Median	95 th	Max
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
<i>Health stereotypes</i>												
Attractive (+)	-0.011 (0.009)	-0.004 (0.007)	-0.006 (0.007)	0.043 (0.025)	-0.008 (0.021)	-0.043* (0.019)	-0.000 (0.009)	-0.001 (0.007)	0.005 (0.007)	0.029 (0.019)	0.023 (0.015)	-0.024 (0.015)
Hearing (+)	-0.017* (0.007)	0.002 (0.004)	0.000 (0.004)	0.008 (0.023)	0.003 (0.015)	0.011 (0.012)	-0.003 (0.007)	0.001 (0.005)	-0.004 (0.005)	0.010 (0.016)	0.021 (0.011)	0.006 (0.009)
Memory (+)	-0.005 (0.007)	-0.007 (0.006)	-0.005 (0.006)	-0.021 (0.021)	0.006 (0.020)	-0.010 (0.015)	-0.016* (0.007)	-0.009 (0.006)	0.000 (0.006)	-0.003 (0.016)	0.010 (0.015)	0.002 (0.013)
Physical Ability (+)	0.000 (0.009)	-0.010 (0.007)	0.001 (0.006)	0.032 (0.025)	-0.031 (0.021)	-0.024 (0.016)	-0.006 (0.009)	-0.001 (0.007)	0.011 (0.006)	0.005 (0.019)	0.013 (0.015)	-0.010 (0.013)
<i>Personality stereotypes</i>												
Adaptable (+)	-0.016 (0.010)	-0.012 (0.008)	-0.004 (0.007)	-0.033 (0.032)	-0.005 (0.023)	0.006 (0.019)	-0.009 (0.010)	0.002 (0.007)	-0.006 (0.007)	-0.004 (0.023)	-0.006 (0.018)	0.001 (0.015)
Careful (-)	0.002 (0.009)	-0.001 (0.007)	-0.007 (0.006)	-0.001 (0.031)	0.034 (0.023)	0.038* (0.018)	0.014 (0.010)	0.003 (0.007)	-0.005 (0.006)	-0.045* (0.020)	-0.025 (0.017)	0.002 (0.014)
Creative (+)	0.030** (0.010)	0.011 (0.007)	0.009 (0.006)	-0.007 (0.033)	-0.029 (0.026)	0.005 (0.018)	0.009 (0.009)	0.003 (0.008)	0.008 (0.007)	-0.022 (0.024)	0.013 (0.017)	0.015 (0.012)
Dependable (-)	0.008 (0.008)	0.007 (0.007)	0.012 (0.006)	-0.020 (0.026)	-0.001 (0.019)	0.023 (0.017)	0.001 (0.008)	-0.016* (0.007)	-0.002 (0.007)	-0.009 (0.019)	0.006 (0.014)	0.019 (0.014)
Personality (+/-)	-0.003 (0.009)	0.007 (0.006)	0.006 (0.006)	-0.007 (0.029)	0.003 (0.022)	0.013 (0.014)	-0.008 (0.008)	0.004 (0.006)	0.004 (0.005)	-0.016 (0.020)	-0.039** (0.015)	-0.017 (0.012)
<i>Skills stereotypes</i>												
Ability to Learn (+)	0.009 (0.010)	0.008 (0.008)	-0.003 (0.007)	-0.009 (0.031)	0.014 (0.025)	0.024 (0.017)	0.012 (0.010)	0.005 (0.008)	0.001 (0.007)	0.018 (0.024)	0.015 (0.019)	0.013 (0.015)
Comm Skills (+/-)	0.004 (0.011)	-0.013 (0.008)	-0.001 (0.007)	0.023 (0.036)	-0.012 (0.029)	-0.052** (0.020)	-0.006 (0.011)	0.001 (0.008)	0.002 (0.007)	0.028 (0.027)	0.007 (0.019)	0.000 (0.016)
Experienced (-)	0.009 (0.007)	-0.000 (0.005)	0.000 (0.005)	-0.039 (0.024)	0.005 (0.015)	-0.013 (0.015)	-0.002 (0.007)	0.002 (0.005)	-0.001 (0.005)	-0.005 (0.017)	0.000 (0.009)	-0.005 (0.010)
Productive (+/-)	-0.028** (0.009)	0.002 (0.007)	0.003 (0.007)	0.030 (0.033)	0.010 (0.020)	-0.001 (0.018)	-0.001 (0.009)	0.006 (0.008)	-0.001 (0.007)	0.032 (0.023)	0.007 (0.015)	0.022 (0.015)
Technology (+)	-0.006 (0.008)	0.009 (0.006)	0.003 (0.006)	-0.009 (0.025)	-0.018 (0.019)	-0.000 (0.015)	0.009 (0.008)	0.005 (0.006)	0.001 (0.006)	-0.009 (0.020)	-0.020 (0.014)	-0.037** (0.013)
N	6,822	6,822	6,827	986	986	987	7,321	7,321	7,330	1,856	1,856	1,861

Note: See notes to Table 4. There are sometimes more observations for the maximum because of ads with small numbers of trigrams for which percentiles could not be accurately calculated. * indicates that the estimate is statistically significant at the 5% level. ** indicates that the estimate is statistically significant at the 1% level.

Appendix Table B2B: Varying the Percentile, Men, Effects on Discrimination Against Older Applicants

Percentile	Middle-Janitor			Middle-Sales			Middle-Security			Old-Janitor			Old-Sales			Old-Security		
	Median	95 th	Max	Median	95 th	Max	Median	95 th	Max	Median	95 th	Max	Median	95 th	Max	Median	95 th	Max
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)	(15)	(16)	(17)	(18)
<i>Health stereotypes</i>																		
Attractive (+)	-0.018 (0.026)	0.017 (0.022)	0.004 (0.019)	0.002 (0.020)	-0.017 (0.017)	-0.020 (0.015)	-0.010 (0.018)	0.019 (0.012)	-0.015 (0.013)	-0.097* (0.044)	0.018 (0.034)	0.019 (0.033)	-0.022 (0.018)	-0.026 (0.014)	0.005 (0.013)	0.005 (0.026)	-0.029 (0.020)	-0.028 (0.017)
Hearing (+)	0.021 (0.019)	-0.007 (0.016)	-0.028* (0.012)	0.011 (0.017)	-0.010 (0.010)	-0.011 (0.008)	0.015 (0.013)	0.020* (0.010)	0.012 (0.007)	0.030 (0.029)	-0.010 (0.023)	-0.013 (0.019)	-0.008 (0.014)	0.000 (0.009)	0.006 (0.009)	0.002 (0.020)	0.012 (0.014)	0.015 (0.014)
Memory (+)	-0.028 (0.024)	-0.019 (0.019)	0.007 (0.015)	0.002 (0.015)	0.030* (0.012)	0.017 (0.011)	-0.022 (0.013)	0.012 (0.010)	0.005 (0.010)	-0.057 (0.032)	0.006 (0.028)	0.001 (0.027)	-0.009 (0.014)	0.006 (0.013)	0.021* (0.010)	-0.019 (0.019)	0.006 (0.017)	0.019 (0.014)
Physical Ability (+)	0.026 (0.021)	0.038* (0.018)	0.029 (0.018)	0.004 (0.017)	0.032* (0.015)	0.000 (0.012)	-0.003 (0.013)	0.022 (0.012)	0.003 (0.011)	0.054 (0.034)	0.002 (0.027)	-0.021 (0.028)	0.040* (0.016)	0.026 (0.015)	0.013 (0.011)	-0.008 (0.016)	-0.002 (0.015)	-0.021 (0.014)
<i>Personality stereotypes</i>																		
Adaptable (+)	0.049 (0.027)	-0.000 (0.020)	-0.003 (0.020)	-0.011 (0.021)	0.032 (0.020)	0.020 (0.015)	0.000 (0.017)	-0.021 (0.011)	-0.006 (0.013)	0.113** (0.039)	-0.001 (0.037)	-0.031 (0.035)	0.000 (0.018)	0.031* (0.016)	0.005 (0.014)	0.005 (0.026)	0.015 (0.018)	0.035 (0.020)
Careful (-)	-0.032 (0.025)	-0.042* (0.020)	-0.025 (0.016)	-0.020 (0.019)	-0.035* (0.015)	-0.007 (0.014)	0.023 (0.016)	-0.016 (0.013)	-0.009 (0.013)	-0.022 (0.036)	-0.014 (0.029)	0.017 (0.027)	-0.007 (0.019)	-0.031* (0.015)	-0.011 (0.013)	0.050* (0.025)	-0.001 (0.018)	-0.014 (0.016)
Creative (+)	-0.032 (0.027)	0.015 (0.023)	-0.026 (0.019)	-0.042 (0.023)	-0.018 (0.016)	-0.009 (0.013)	-0.008 (0.018)	-0.026* (0.013)	-0.020 (0.014)	-0.033 (0.048)	0.035 (0.036)	-0.040 (0.029)	-0.021 (0.021)	-0.008 (0.014)	-0.007 (0.009)	-0.044 (0.026)	0.011 (0.023)	-0.010 (0.020)
Dependable (-)	0.004 (0.024)	0.020 (0.016)	0.012 (0.013)	0.017 (0.017)	0.011 (0.015)	0.018 (0.013)	-0.004 (0.015)	0.004 (0.009)	0.000 (0.009)	0.004 (0.031)	0.016 (0.024)	-0.003 (0.023)	-0.003 (0.018)	-0.001 (0.014)	0.006 (0.011)	-0.011 (0.021)	0.006 (0.014)	0.017 (0.014)
Personality (+/-)	0.039 (0.026)	0.019 (0.017)	0.026 (0.018)	0.025 (0.017)	0.021 (0.014)	0.009 (0.009)	-0.003 (0.006)	-0.014 (0.006)	-0.021 (0.006)	0.071 (0.040)	0.035 (0.033)	0.055 (0.029)	0.020 (0.017)	-0.002 (0.012)	-0.008 (0.009)	0.034 (0.026)	-0.014 (0.020)	-0.032 (0.019)
<i>Skills stereotypes</i>																		
Ability to Learn (+)	-0.008 (0.025)	0.004 (0.025)	-0.013 (0.019)	0.006 (0.023)	-0.045* (0.018)	-0.015 (0.014)	-0.000 (0.019)	-0.009 (0.015)	0.008 (0.014)	-0.027 (0.037)	0.027 (0.036)	0.006 (0.030)	-0.009 (0.019)	-0.022 (0.017)	-0.017 (0.013)	-0.031 (0.028)	-0.001 (0.023)	0.012 (0.019)
Comm Skills (+/-)	-0.028 (0.031)	0.002 (0.033)	0.048* (0.022)	-0.019 (0.022)	0.003 (0.020)	-0.001 (0.013)	-0.013 (0.022)	0.010 (0.015)	0.012 (0.014)	-0.009 (0.053)	-0.068 (0.053)	0.033 (0.040)	-0.007 (0.018)	0.015 (0.019)	0.016 (0.013)	-0.040 (0.029)	0.004 (0.027)	0.019 (0.021)
Experienced (-)	0.039 (0.020)	0.033** (0.012)	0.020* (0.009)	-0.006 (0.018)	0.014 (0.010)	0.013 (0.009)	-0.002 (0.015)	-0.012 (0.008)	-0.009 (0.010)	0.015 (0.032)	0.042* (0.019)	0.036* (0.015)	-0.020 (0.015)	-0.007 (0.008)	-0.012 (0.009)	0.009 (0.020)	0.003 (0.011)	0.011 (0.010)
Productive (+/-)	-0.037 (0.029)	-0.049* (0.021)	-0.023 (0.017)	0.021 (0.022)	-0.022 (0.016)	-0.019 (0.015)	-0.005 (0.019)	0.014 (0.013)	0.033* (0.013)	-0.036 (0.040)	-0.036 (0.037)	0.009 (0.033)	0.011 (0.019)	-0.002 (0.012)	0.000 (0.013)	-0.002 (0.028)	0.005 (0.020)	-0.001 (0.021)
Technology (+)	0.022 (0.024)	0.011 (0.014)	-0.026 (0.017)	0.026 (0.016)	0.004 (0.012)	0.002 (0.011)	0.030 (0.016)	0.021* (0.009)	0.002 (0.012)	-0.001 (0.037)	0.034 (0.026)	0.000 (0.027)	0.014 (0.015)	-0.009 (0.013)	-0.012 (0.011)	0.037 (0.021)	-0.007 (0.016)	-0.028 (0.019)
N	311	311	311	1,612	1,612	1,612	954	954	956	318	318	318	1,680	1,680	1,680	932	932	932

Note: See notes to Table 4 and Appendix Table B2A. * indicates that the estimate is statistically significant at the 5% level. ** indicates that the estimate is statistically significant at the 1% level.

Appendix Table B3A: Refinements to the Coding of Experience, Women, Effects on Discrimination Against Older Applicants

	Middle-Admin			Middle-Sales			Old-Admin			Old-Sales		
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
Experience CS Score	-0.000 (0.005)	-0.001 (0.005)	-0.002 (0.005)	0.005 (0.015)	0.011 (0.015)	0.013 (0.016)	0.002 (0.005)	0.004 (0.005)	0.004 (0.005)	0.000 (0.009)	0.006 (0.010)	0.007 (0.010)
No Experience Needed		-0.002 (0.015)	-0.002 (0.015)		-0.041 (0.038)	-0.043 (0.038)		-0.029 (0.017)	-0.029 (0.017)		-0.032 (0.025)	-0.033 (0.025)
Experience CS Score × No Experience Needed		0.017 (0.015)	0.017 (0.015)		-0.015 (0.030)	-0.014 (0.030)		0.020 (0.015)	0.020 (0.015)		0.006 (0.027)	0.007 (0.027)
1-2 Years Experience			0.053 (0.039)			-0.051 (0.059)			0.006 (0.033)			-0.032 (0.056)
Experience CS Score × 1-2 Years Experience			-0.003 (0.024)			0.014 (0.065)			-0.012 (0.022)			-0.038 (0.039)
N	6822	6822	6822	986	986	986	7321	7321	7321	1856	1856	1856

Note: Table reports estimated marginal effects. For each age-occupation group, the first column estimates equation (10), but reports only the coefficient for the 95th percentile of Experience. In the second column, we include a dummy the ad indicating no experience was needed and an interaction between the no experience needed dummy and the Experience CS score. In the third column, we add a dummy for only requiring one to two years of experience and the interaction between the low experience dummy and the Experience CS score (equation (12)). Standard errors are clustered at the job ad level. * indicates that the estimate is statistically significant at the 5% level. ** indicates that the estimate is statistically significant at the 1% level.

Appendix Table B3B: Refinements to the Coding of Experience, Men, Effects on Discrimination Against Older Applicants

	Middle-Janitor			Middle-Sales			Middle-Security			Old-Janitor			Old-Sales			Old-Security		
	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)	(13)	(14)	(15)	(16)	(17)	(18)
Experience CS Score	0.033** (0.012)	0.034** (0.012)	0.033** (0.012)	0.014 (0.010)	0.012 (0.010)	0.011 (0.010)	-0.012 (0.008)	-0.010 (0.009)	-0.009 (0.009)	0.042* (0.019)	0.041* (0.019)	0.041* (0.019)	-0.007 (0.008)	-0.007 (0.008)	-0.008 (0.008)	0.003 (0.011)	-0.002 (0.012)	-0.004 (0.013)
No Experience Needed		-0.021 (0.034)	-0.022 (0.033)		0.014 (0.024)	0.006 (0.025)		-0.020 (0.027)	-0.018 (0.027)		-0.034 (0.057)	-0.033 (0.056)		0.003 (0.020)	-0.000 (0.020)		0.033 (0.039)	0.034 (0.039)
Experience CS Score × No Experience Needed		0.070 (0.060)	0.064 (0.058)		0.010 (0.023)	0.014 (0.024)		0.014 (0.026)	0.013 (0.026)		0.097 (0.076)	0.098 (0.077)		0.020 (0.023)	0.024 (0.023)		0.041 (0.038)	0.040 (0.038)
1-2 Years Experience			-0.054 (0.046)			0.195* (0.096)			-0.029 (0.037)			-0.012 (0.124)			0.051 (0.035)			0.011 (0.077)
Experience CS Score × 1-2 Years Experience			-0.038* (0.019)			-0.045 (0.030)			-0.010 (0.027)			-0.011 (0.073)			-0.049* (0.020)			0.061 (0.061)
N	311	311	311	1,612	1,612	1,610	954	954	953	318	318	318	1,680	1,680	1,680	932	932	931

Note: See note to Appendix Table B3A. * indicates that the estimate is statistically significant at the 5% level. ** indicates that the estimate is statistically significant at the 1% level.

Online Appendix C: Technical Appendix

This appendix provides explanation and code for our core methods.

1. Extracting the Trigrams

We begin by identifying all the trigrams in a job ad using the following python code. This proceeds in a number of steps.

First, the code below reads the file “CallBack.csv” (which contains the text of all the ads saved as part of NBB) containing all the ads and does some manipulation in preparation for further processing.ⁱ

```
dfads = pd.read_csv("D:\RAWorkAgeDiscrim\ML\CallBack.csv")
dfads.head()

# Substitute blank spaces with x so the program runs later

dfads["Body"] = dfads["Body"].replace(" ", "x", regex=True)
dfads["Body"] = dfads["Body"].replace(np.nan, "x", regex=True)

# Dropping duplicates

dfads=dfads.drop_duplicates(subset="jobid", keep="first")

#Fixing index
dfads.index = np.arange(0,len(dfads))

# Keep only job id and body
match = dfads[["jobid", "Body"]]
```

Second, we create the program *review_to_words* that eliminates spaces, caps, punctuation, and stop words.

ⁱ A user of this code will of course have to change the file names and folders.

```
def review_to_words(add):
    # Function to convert a raw review to a string of words
    # The input is a single string (a raw movie review), and
    # the output is a single string (a preprocessed movie review)
    #
    # 1. Remove HTML (I don't need this part already cleaned html)
    #review_text = BeautifulSoup(raw_review).get_text()
    #
    # 2. Remove non-letters
    letters_only = re.sub("[^a-zA-Z]", " ", add)
    #
    # 3. Convert to lower case, split into individual words
    words = letters_only.lower().split()
    #
    # 4. In Python, searching a set is much faster than searching
    # a list, so convert the stop words to a set
    stops = set(stopwords.words("english"))
    #
    # 5. Remove stop words
    meaningful_words = [w for w in words if not w in stops]
    #
    # 6. Join the words back into one string separated by space,
    # and return the result.
    return " ".join(meaningful_words))

num_adds = match["Body"].size
```

Third, we run the program to clean each ad. This means that we are eliminating symbols, non-English characters, punctuation, and stop words. The objective is to keep only relevant words that convey meaning potentially related to stereotypes.

```
clean_add = review_to_words( match["Body"][2] )
print clean_add
print match["Body"][2]

print "Cleaning and parsing the training set movie reviews..."
clean_train_adds = []
for i in xrange( 0, num_adds ):
    # If the index is evenly divisible by 1000, print a message
    if ( i+1)%1000 == 0 ):
        print "Review %d of %d\n" % ( i+1, num_adds )
    clean_train_adds.append( review_to_words( match["Body"][i] ) )
match["Body_Clean"] = clean_train_adds
```

Fourth, we import *nltk* to make trigrams for each ad.

```
# Creating trigrams
#Note that you need to download> nltk.download("punkt")

from nltk.util import ngrams
from nltk import word_tokenize
```

Finally, in the code below, in the first line, using `ngrams(word_tokenize())`, we separate all the trigrams in each ad. The line `match = match[["jobid", "trigram"]]` separates trigrams by job ad, creating a row with each trigram separated by commas. The following line outputs them into an csv file with a column for the trigrams in each ad.

```
match["trigram"] = match["Body_Clean"].apply(lambda row: list(ngrams(word_tokenize(row),
3)))

match = match[["jobid", "trigram"]]

match.to_excel("D:\\RAWorkAgeDiscrim\\SScoresMatch\\matchtrigram.xlsx")
```

Appendix Figure C1, Panels A and B, show the files created at these two steps, with the trigrams separated by commas (Panel A) and the trigrams then split into columns (Panel B).

Appendix Figure C1: Trigrams from Job Ads

Panel A. Raw Output Panel						B. Arranged in Columns					
A	B	C	D	E	F	A	B	C	D	E	F
	jobid	trigram					jobid	trigram			
0	4826529596	[(music, fest, time), (fest, time, year), (time, year, right)]				0	4826529596	music fest fest time time year year right			
1	4833493607	[(flower, patch, currently), (patch, currently, seeking q)]				1	4833493607	flower patch patch curr currently seeking q			
2	4833543304	[(seeking, career, change), (career, change, looking ge)]				2	4833543304	seeking ca career cha change lo looking ge			
3	4835083297	[(interior, decorator, trainee), (decorator, trainee, furniture)]				3	4835083297	interior di decorator trainee fu furniture			
4	4835197759	[(opportunity, become, key), (become, key, member c)]				4	4835197759	opportuni become k key memt member c			
5	4841996184	[(furniture, warehouse, currently), (warehouse, currently, seeking hi)]				5	4841996184	furniture warehouse currently seeking hi			
6	4843542824	[(looking, person, least), (person, least, years den)]				6	4843542824	looking pe person le least year years den			
7	4843588254	[(seeking, exceptional, hotel), (exceptional, hotel, front desk)]				7	4843588254	seeking e: exception hotel fron front desk			
8	4843935707	[(local, retail, store), (retail, store, seeking re)]				8	4843935707	local retai retail stor store seek seeking re			
9	4844967381	[(yale, enforcement, services), (enforcement, services, seeking ui)]				9	4844967381	yale enfor enforcem services si seeking ui			
10	4845013578	[(dealership, parts, counter), (parts, counter, position n)]				10	4845013578	dealership parts cour counter pi position n			
11	4845385448	[(seasonal, part, time), (part, time, front)]				11	4845385448	seasonal part time time front front desk			

2. Calculating the Semantic Similarity between Words using Word2Vec

Word2Vec is an algorithm that calculates vectors for a vocabulary and estimates the relationships between different words. The steps to use the *Word2Vec* algorithm are:

1. Download all Wikipedia articlesⁱⁱ (or the corresponding corpus used) and store the information on a computer's hard drive.
2. Read all the articles and separate them into paragraphs and sentences.
3. Clean text: drop all HTML code, punctuation, and symbols; and substitute caps for lowercase.
4. The python module *gensim* then generates all the vectors.
5. Identify the words used together in sentences and paragraphs and count the number of times that a pair of words appear together, for all the words in the corpus.
6. Estimate the probability that two words appear together.

ⁱⁱ The database (*Wikipedia dump*) articles are available at <https://dumps.wikimedia.org/enwiki/>.

7. Create 200-dimensional vectors for each word. The 200 dimensions are a measure of the vocabulary that describes the characteristics of each word. The method simplifies all the information using hidden layers, and instead of having all the words as a context, we only need 200 weights. In this step, similar words (because they share the same context) have similar vectors; as a result, the cosine similarity score (CSS) between them will be closer to one.
8. Store the vectors in a matrix for future use.

The code to do this is now described in more detail. All the coding is in Python 2.7. Use “—future—” to make compatible all the coding to Python 3.x.

This first section of the code imports all the modules needed to run the code correctly, and takes some other initial steps.

gensim: Train the model on Wikipedia. For information on how the *gensim* program works, please see <https://radimrehurek.com/gensim/>.ⁱⁱⁱ

logging: This has multiple purposes. Logging retains information in some parts of the code, like usernames, passwords, the directory where the files are, etc. It can print part of the process, give you warnings, and restart the kernel (python) and other functions. All these errors and warnings are stored and printed. It is not relevant to the algorithm that we are using.

multiprocessing: This is used to scan the computer’s number of processors (workers in python) and use the maximum number to compute the estimations of *Word2Vec* more quickly.

os: This is necessary to navigate through the folder and files on one’s computer.

re: This includes operations like detecting caps, symbols, and numbers and dropping them, adding them, replacing them, etc.

sys: This module provides access to some variables used or maintained by the interpreter and functions that interact strongly with the interpreter. It is always available. In our code it is used to read the pathname of the operating system.

```
from __future__ import print_function
import gensim
import logging
import multiprocessing
import os
import re
import sys
```

nltk is imported specifically to tokenize all the paragraphs and sentences in Wikipedia’s articles.

nltk.data.load(“tokenizers/punkt/english.pickle”): This sets English as the language to analyze the text.

ⁱⁱⁱ In our view, this is the best source for understanding the *Word2Vec* algorithm. It both provides code and references to many related articles.

time: This times the process.

logging.basicConfig: This sets up the format, and sets alerts, warnings, and errors (coded as levelname).

```
import nltk.data
tokenizer = nltk.data.load("tokenizers/punkt/english.pickle")
from time import time

logging.basicConfig(format='%(asctime)s : %(levelname)s : %(message)s',
                    level=logging.INFO)
```

We define a function that cleans all the text in Wikipedia's articles, eliminating punctuation, some HTML code, and keeping only the relevant text.

def: This indicates that we are defining a new function.

re.compile: This creates a pattern of the symbols to search for and match in the html code.

re.sub: This part of the code looks for the compiled pattern created in *re.compile*, and replaces it with "blank." If the pattern isn't found, it returns the desired text.

```
def cleanhtml(raw_html):
    cleanr = re.compile('<.*?>')
    cleantext = re.sub(cleanr, "", raw_html)
    return cleantext
```

Next, we define the Windows path where all the Wikipedia files are saved (usually on the hard drive). The program will ask for the directory at the beginning. After that, it extracts all the articles from a "dump" file available in *Wikimedia dump* (see footnote iii).

```
class MySentences(object):
    def __init__(self, dirname):
        self.dirname = dirname
```

Next, we put together some functions to read all the articles to clean paragraphs and sentences.

for root: This is a loop through the directories and files on the folder where the Wikipedia files are saved.

for filename in files: This loops through the files.

sline: This separates the articles into sentences.

cleanhtml: This cleans the sentences of each article.

```
def __iter__(self):
    for root, dirs, files in os.walk(self.dirname):
        for filename in files:
            file_path = root + '/' + filename
            for line in open(file_path):
                sline = line.strip()
                if sline == "":
                    continue
                rline = cleanhtml(sline)
```

Once the sentences are clean, they are tokenized (split) into words.

Tokenizer.tokenize(): split sentences into words.

The rest of the lines replace caps with lowercase and drop symbols.

```
tokenized_line = ''.join(tokenizer.tokenize(rline))
    is_alpha_word_line = [word for word in
                          tokenized_line.lower().split()
                          if word.isalpha()]
    yield is_alpha_word_line

if __name__ == '__main__':
    if len(sys.argv) != 2:
        print ("Please use python train_with_gensim.py data_path")
        exit()
    data_path = sys.argv[1]
    begin = time()
```

We now explain the *Word2Vec* model, which transforms all the words and sentences into vectors. We use *gensim* (a python module) to do this. Our parameters are as follows:

size = 200 dimensions. Generally, to analyze semantic similarity with massive databases like Wikipedia, the recommended size is between 100 and 150 dimensions (nodes); see Pennington et al. (2014). The more nodes, the more precise the model will be. We picked 200 nodes to increase the precision of the semantic similarity; we did not exceed 200 because it is computationally cumbersome.

window = 10. The window defines how many words before and after the target word are used to estimate the probability that they are together in the same context. It is recommended in user forums (at <https://stackoverflow.com/>) to use 10 words (the default). If one selects fewer words, then the CSS will be more strict. Only words that are very close in a sentence are related.

workers = `multiprocessing.cpu_count()`. This defines how many cores of the processor are used to calculate everything. "`multiprocessing.cpu_count()`" identifies the number of cores in the computer and uses it; the code results in using all the cores available in the computer to calculate the vectors faster.

```
sentences = MySentences(data_path)
model = gensim.models.Word2Vec(sentences,
                                size=200,
                                window=10,
                                min_count=10,
                                workers=multiprocessing.cpu_count())
model.save("data/model/word2vec_gensim")

model.wv.save_word2vec_format("D:\RAWorkAgeDiscrim\Wikicorpus\wikiextractor\word2vec_org",
                              "D:\RAWorkAgeDiscrim\Wikicorpus\wikiextractor\vocabulary", binary=False)
```

The last line in the code estimates how many seconds it took to calculate all the vectors.

```
end = time()
print ("Total procesing time: %d seconds" % (end - begin))
```

The calculations are saved as a large matrix. Each row of the matrix corresponds to a word from Wikipedia, and the columns include the weights on the 200 vectors that corresponding to words. The matrix is saved in an *npv* file that can only be read in python.

3. Merging with Trigrams from Job Ads

We create an Excel sheet ("*3gram*") with the corresponding CSS of all the trigrams in the job ads with all of the stereotypes (Appendix Figure C2).^{iv}

Appendix Figure C2. Example of the Database of CSS's between Trigrams and Stereotypes

Unnamed: 0 Word	productive	adaptable	warm persor communicat	creative	ability learn	experienced dependable	attractive	memory	careful	hearing	technology	physically at	personality
0 catalogs coordinate shipping	0.08990942	0.05313969	-0.19760552	0.20634335	0.06176708	0.07348558	-0.09472074	0.09854512	-0.02646285	0.02381901	0.06710053	-0.16429107	0.23960878
1 sales southwest ranches	0.10536993	-0.04885759	-0.03147481	-0.05422942	-0.05438927	-0.11572261	0.01313831	0.01774774	0.01193705	-0.17657453	-0.18822904	-0.1800341	0.10438366
2 prospective clients success	0.16818222	0.12669513	0.10045204	0.32829291	0.23495799	0.26456618	0.17890905	0.22814527	0.25742427	-0.01805643	0.19022837	0.11348414	0.15885344
3 appointment scheduling writing	0.03171875	-0.04546606	0.14023437	0.25051039	0.34466651	0.12218836	0.02775814	0.11804051	-0.06377374	0.08669132	0.20935424	0.27556646	0.07871884
4 simon plaza williston	-0.19512244	-0.11748193	-0.05212216	-0.19663957	0.04461493	-0.19098368	-0.2260315	-0.11025863	-0.02714494	0.02908025	-0.13633309	-0.03011486	-0.05571298
5 skills transportation needed	0.34593928	0.32764688	0.04814171	0.72389758	0.24205811	0.53654891	0.17179905	0.38766405	0.21872398	0.09463752	0.3626931	0.04849161	0.43690085
6 would include microsoft	0.05507252	0.1282331	-0.07625295	0.12048694	0.203337	0.18017448	0.15983821	0.0466962	0.07154812	-0.00294341	0.15222321	0.01580893	0.17686015
7 fit position meet	0.01135917	0.21331298	0.14873748	0.19418238	0.00165645	0.29324073	0.02988006	0.15057229	0.12077843	0.04659567	0.03246775	0.17623547	0.03006035
8 duties within assigned	0.15287887	0.06351158	-0.08917952	0.19239633	-0.01984116	0.07527398	0.01833964	0.10259384	-0.10679894	-0.03612029	0.11287006	-0.06852864	-0.03495055
9 essential interested applying	0.40528563	0.37229681	0.08665893	0.38399914	0.28855851	0.38470435	0.16601919	0.21265782	0.43854371	0.01634677	0.46559557	0.03277518	0.25115901
10 requirements conditions listed	0.09562268	0.22609578	0.00514583	0.19581731	-0.10460529	0.10000484	0.11359353	0.06344416	0.05290517	0.01450045	0.17482537	0.00580651	0.18993343
11 team html look	-0.01515979	0.14693253	0.16303231	0.15821129	0.16901663	0.27929473	-0.02010506	0.10175705	0.15552315	0.11618007	0.02925275	0.03741275	0.14442667
12 group busy medical	0.15953976	0.05862157	-0.01099385	0.24548081	0.19480364	0.03549661	0.05429187	0.14650725	0.07932108	-0.04371284	0.17161372	0.06139317	0.21899821
13 data entry south	-0.01073593	0.03443855	-0.06379509	0.16544062	-0.09061332	0.08198788	-0.15355833	0.00821663	-0.04550007	0.13554764	-0.01189012	-0.09298855	0.27372527
14 environment multiple daily	0.20707737	0.3114644	0.29219025	0.40150142	0.28879637	0.31495175	0.06950984	0.10042349	0.17320518	0.17719334	0.20981269	0.02840904	0.35128736
15 monday saturday right	-0.15820372	-0.11020786	0.09490361	-0.04754947	-0.11052027	0.05618757	-0.15060109	-0.07671998	-0.07000691	-0.01794079	-0.02857125	0.18608308	-0.17914858
16 seeking friendly patient	0.13009594	0.15739988	0.27222458	0.24603693	0.04549244	0.33473244	0.16545109	0.21027303	0.25369608	0.10025571	0.34812698	0.28539941	0.08464491
17 daily evening events	-0.1608668	-0.0626367	0.17546733	0.09843986	0.02043072	0.02739242	-0.02296785	-0.06856505	-0.02023997	0.03499395	-0.0318891	0.15581027	-0.0547977
18 individuals provide security	0.1268528	0.29196921	-0.00695784	0.42222661	0.09578838	0.3035934	0.08727432	0.08990557	0.11403897	0.12934859	0.13164648	0.12627788	0.3121103

We then want to array these by job ad. To do this we match the trigrams in each ad with the values in sheet *3gram*, using the function *vlookup* in Excel. Once we have the CSS of the trigrams for each job ad (Appendix Figure C3), we can easily estimate the mean, the variance, the median, and the 90th and 95th percentiles.

^{iv} The transition of the vectors used to compute the CS scores from words to trigrams is explained in the paper.

Figure C3. Example of Job Ads with CSS for each Trigram

jobid	1	2	3	4	5	6	7	8	9	10	11	12	13	14
4826529596	-0.10024114	-0.09708729	-0.01844186	0.01448191	-0.03113504	0.01227904	-0.028702	0.00467881	-0.04675245	-0.04064595	-0.01487927	-0.07361431	-0.05576007	0.01312138
4833493607	-0.00617863	0.04613127	0.13083598	0.13241211	0.07699008	0.00921701	0.03383891	0.00771914	0.06467864	0.16539715	0.29615444	0.27665299	0.14728437	0.04790819
4833543304	0.21578708	0.17278934	0.03393763	-0.0699803	-0.21475951	-0.18009485	-0.09426907	-0.0203431	0.02943952	0.09283339	0.23393318	0.1111477	0.10658752	0.0616859
4835083297	0.03589931	0.07511814	0.09851844	0.07501265	0.10140617	0.04791774	0.05062216	0.15721832	0.24757797	0.19007869	0.13659912	0.10788237	0.13320187	0.10031367
4835197759	0.22598884	0.16380179	0.04292243	0.04810782	-0.00053314	0.06695009	-0.04439499	0.12912951	0.13929945	0.17689294	0.08671943	0.16388877	0.18513478	0.1695627
4841996184	0.03220937	0.04563214	0.17953841	0.27428922	0.31797147	0.32621011	0.26503557	0.05160114	0.03055433	0.0673307	0.2169556	0.18767482	0.22436081	0.2372003
4843542824	0.11018354	0.09392221	0.08907165	0.11684136	0.29699579	0.34429896	0.2828511	0.29532799	0.24810651	0.24442217	0.13075891	0.16802828	0.16100369	0.16328786
4843588254	0.08232513	-0.11876513	-0.2668415	-0.07415906	0.06851102	0.2120795	0.16378568	0.17194752	0.08303604	0.14349811	0.19362704	0.18216772	0.24480003	0.28867501
4843935707	0.01724329	-0.00311806	-0.00311805	0.0984218	0.09500656	0.07983962	0.13013262	0.12435132	0.01721883	-0.0234438	0.05834471	0.19795772	0.24787803	0.1176051
4844967381	0.06123802	0.08094908	0.04454608	0.01026424	-0.05688218	0.02195812	0.02264082	0.07050962	0.06763348	0.07946321	0.13165614	0.1298659	0.10242955	0.08656864
4845013578	-0.02677057	-0.00638406	-0.05178473	-0.02342796	-0.09080653	-0.03263257	0.03076433	0.08058264	0.04988318	0.00063334	0.13504805	0.16772181	0.19037443	0.05785627
4845385448	0.16387667	-0.07936481	-0.16221321	-0.21648592	-0.01154722	0.18576795	0.22141823	0.28458473	0.10574394	0.12344759	-0.00180374	0.15091908	0.24716361	0.33783776
4846715732	0.00396486	0.03477761	0.00195647	-0.14778571	-0.16713858	-0.15529579	0.1444006	0.2309681	0.3593688	0.23717786	0.25128782	0.11241271	0.06510139	-0.08271141
4847961364	-0.01452504	0.05434846	0.24676783	0.22714891	0.20228805	0.04652498	-0.08945298	-0.16283928	-0.1491413	0.02857964	0.14077154	0.28889999	0.16714802	0.1566406
4849587887	-0.12659535	-0.13430925	-0.18079892	-0.15327126	-0.12263446	-0.04791224	0.01008615	0.09241977	0.09241977	0.09241977	-0.0060811	-0.04221516	0.02635401	0.06603011
4849822816	0.12484151	0.10832646	0.12743169	0.05535002	-0.00200086	0.05923638	0.05923638	0.06952	-0.05810596	-0.05242701	-0.05864092	-0.05228057	0.0044537	-0.08131877
4853496101	0.02508972	0.15762445	0.08941705	0.10315267	-0.06815395	-0.00076564	-0.03403388	0.0524486	-0.05848509	-0.0565788	-0.07487626	-0.02046918	0.1047399	0.17303221
4853950964	0.31863141	0.10225229	0.05329071	0.02865437	-0.04688453	-0.14647791	-0.15236194	-0.02816193	-0.04039276	-0.08843983	-0.08673836	0.0506306	0.23228753	0.21930367
4854071360	0.02147976	-0.01066627	0.14758992	0.18806379	0.13778502	0.03352794	0.15261887	0.20403506	0.22562669	0.09278277	0.2243678	0.11057214	0.25189361	0.10366898
4854184121	0.13234863	0.11073428	0.1277038	-0.02811816	-0.10372508	-0.20075628	-0.10033982	0.01887384	0.12432531	0.09552225	0.15402076	0.17795062	0.23658381	0.23422804
4854184958	0.01950624	0.02677039	0.04314319	0.06319994	-0.00332793	-0.05309774	-0.137986	-0.10930039	-0.07223731	-0.06222705	-0.09666509	-0.1154443	-0.06697967	0.01219462

4. A Narrower Corpus

We used another algorithm to train our model on a subset of Wikipedia articles – articles related to age, stereotypes, and labor markets. We used a python module called *scrapy* to facilitate this task. We use the following steps:

1. First, we defined a set of Wikipedia articles related to ageism and labor markets, using the following articles: ageism, aging, discrimination, workforce, job, employment, stereotype, sexism, recruitment, skill_(labor), and human_capital.
2. Second, the algorithm scrapes all the paragraphs and sentences from this set of articles.
3. Third, algorithm creates “spiders” that click all the links in these articles, assembles those articles, and scrapes them.
4. Fourth, it repeats this on the new set of articles. (We chose to use two levels.)
5. Finally, once we have this new, smaller corpus, we apply the *gensim* algorithm to train a new model. We refer to this model as the “scrapy-spider” model.

Below, we discuss the code used in these steps.

First, we import all the modules that we are going to use. The most important module is *scrapy*, which scrapes articles from Wikipedia.

```
import re
import logging
from tqdm import tqdm
import scrapy
from scrapy.crawler import CrawlerProcess
```

We name the process.

```
class discrim5(scrapy.Spider):
    name = “discrim5”
```

To construct our smaller corpus, we want to restrict the clicks only to those in the Wikipedia domain. Thus, if the spiders find a link outside Wikipedia, they ignore it.

```
allowed_domains = ["en.wikipedia.org"]
```

We define the start urls (all related to ageism and labor markets).

```
start_urls = [  
    "https://en.wikipedia.org/wiki/Ageism",  
    "https://en.wikipedia.org/wiki/Ageing",  
    "https://en.wikipedia.org/wiki/Discrimination",  
    "https://en.wikipedia.org/wiki/Workforce",  
    "https://en.wikipedia.org/wiki/Job",  
    "https://en.wikipedia.org/wiki/Employment",  
    "https://en.wikipedia.org/wiki/Stereotype",  
    "https://en.wikipedia.org/wiki/Sexism",  
    "https://en.wikipedia.org/wiki/Recruitment",  
    "https://en.wikipedia.org/wiki/Skill_(labor)",  
    "https://en.wikipedia.org/wiki/Human_capital",  
]
```

We also define some settings. The most important are:

feed format: This specifies that the output will be in csv format.

depth limit: This defines how many times the spiders will click links on the website (article) – or the number of levels we use.

```
custom_settings = {  
    'LOG_LEVEL': logging.WARNING,  
    'FEED_FORMAT': 'csv',  
    'FEED_URI': 'discrim5.csv',  
    'DEPTH_LIMIT': 2,  
}
```

In the next section, we define the codes for parsing all the articles and keeping only essential words (no symbols or HTML code).

parse (parsing): Reads all the html code and extracts the title of the article and the main text.

next_page: This reads all the links in the article.

remove_html_tags: This removes all the html code (like head, body, etc.).

```
def parse(self, response):
    for quote in response.css('.mw-parser-output > p'):
        yield {
            'quote': quote.extract(),
            'title': response.css("#firstHeading").extract(),
        }

    for next_page in response.css('div.mw-parser-output > p > a'):
        yield response.follow(next_page, self.parse)

def __remove_html_tags__(self, text):
    html_tags = re.compile('<.*?>')
    return re.sub(html_tags, "", text)
```

Here we just run all the processes and programs that we define above.

```
process = CrawlerProcess()
process.crawl(discrim5)
process.start()
```

Next, we import *pandas*, a spreadsheet module to save all the scraped text to spreadsheets.

```
import pandas as pd
dfquotes = pd.read_csv("discrim5.csv")
dfquotes.head()
```

We clean the articles further (scrapy-spider only eliminates tags) using *BeautifulSoup* (another python module). This step drops any symbol or HTML code that was not identified by *scrapy*. The code includes:

num_articles: This command is used to identify the number of articles. It is used in the next loop when we clean the articles.

```
#Cleaning

from bs4 import BeautifulSoup

num_articles = dfquotes["quote"].size
```

We loop to clean the text using *BeautifulSoup*.^v The loop cleans the body text (*clean_p*) and the titles (*clean_title*).

Database: This matches the cleaned titles with the cleaned body.

^v We have to do this for scrapy-spider because when we scrape directly from the online articles, html codes are included. They are already removed from the Wikipedia dump for the entire corpus.

```
clean_p = []
clean_title = []
for i in tqdm(range( 0, num_articles )):
    clean_p.append(BeautifulSoup(dfquotes["quote"].iloc[i]).get_text())
    clean_title.append(BeautifulSoup(dfquotes["title"].iloc[i]).get_text())

dfquotes["title_c"] = clean_title
dfquotes["body_c"] = clean_p

database = dfquotes[["title_c", "body_c"]]
```

Once we have the clean articles in a spreadsheet, we use *nunique()* to count the number of articles.

```
# Number of articles
database["title_c"].nunique()
```

The following code saves all the data.

```
database.to_excel("D:\\RAWorkAgeDiscrim\\SScoresMatch\\discrim.xlsx")
database.to_csv("D:\\RAWorkAgeDiscrim\\SScoresMatch\\discrim.csv")

#database=database.drop_duplicates(subset="title_c", keep="first")
#database.to_excel("D:\\RAWorkAgeDiscrim\\SScoresMatch\\titles.xlsx")
```

The next part of the code (another file) trains the model. The code is very similar to the code used for training with the whole Wikipedia corpus. However, instead of reading all the Wikipedia dump, we read from the file we created in the last step (discrim.csv).

```
dfads = pd.read_csv("D:\RAWorkAgeDiscrim\SScoresMatch\discrim.csv")
dfads.head()
dfads["body_c"] = dfads["body_c"].replace(" ", "x", regex=True)
dfads["body_c"] = dfads["body_c"].replace(np.nan, "x", regex=True)

# Load the punkt tokenizer
tokenizer = nltk.data.load('tokenizers/punkt/english.pickle')

# Define a function to split a review into parsed sentences
def review_to_sentences( review, tokenizer, remove_stopwords=False ):
    # Function to split a review into parsed sentences. Returns a
    # list of sentences, where each sentence is a list of words
    #
    # 1. Use the NLTK tokenizer to split the paragraph into sentences
    raw_sentences = tokenizer.tokenize(review.strip())
    #
    # 2. Loop over each sentence
    sentences = []
    for raw_sentence in raw_sentences:
        # If a sentence is empty, skip it
        if len(raw_sentence) > 0:
            # Otherwise, call review_to_wordlist to get a list of words
            sentences.append( review_to_wordlist( raw_sentence, \
                remove_stopwords ))
    #
    # Return the list of sentences (each sentence is a list of words,
    # so this returns a list of lists
    return sentences

sentences = [] # Initialize an empty list of sentences

print ("Parsing sentences from training set")
for review in tqdm(dfads["body_c"]):
    sentences += review_to_sentences(review, tokenizer)

# Training the model

model = gensim.models.Word2Vec(sentences, size=200, window=10,
                               min_count=5, workers=multiprocessing.cpu_count())

model.init_sims(replace=True)

model.save("D:\RAWorkAgeDiscrim\Wikicorpus\wikiextractor\Spiders\model_spiders")
```

We also tried the Google News corpus. We did not scrape or train this model because it is available in gensim. We need to import it using the following code:

```
import gensim
from gensim.models import Word2Vec
import gensim.downloader as api
# This files download the google news model (run only if you don't have it)

#import gensim.downloader as api
#This part of the code retrieve the google news model.
model = api.load('word2vec-google-news-300')
```

We decided not to use Google News corpus because it is not fully clean. It includes stop words like “the,” “not,” “there,” etc. Also, symbols like “_” and “\$.” Therefore we discarded the model because it is not comparable with the other ones.

5. Comparing Different Models

In this section we discuss comparing our two models. (This could be applied easily to a model trained on another corpus.) We compare them based on examining what words they rank as most related to a stereotype. In the example below, we ask the model to give us the 10 most similar words to the stereotype “attractive,” based on the CSS. (This could be modified for any number, of course.)

First, we use the full Wikipedia model:

```
model =
Word2Vec.load("D:\RAWorkAgeDiscrim\Wikicorpus\wikiextractor\word2vec_gensim")

#Size of the model:
print (model.wv.syn0.shape)

#Test
model.most_similar("attractive")
Output:
[('unattractive', 0.7458347678184509),
 ('appealing', 0.6965897679328918),
 ('desirable', 0.6877966523170471),
 ('unappealing', 0.6699395179748535),
 ('alluring', 0.6611822843551636),
 ('enticing', 0.6471317410469055),
 ('ideal', 0.6289623975753784),
 ('advantageous', 0.6268125772476196),
 ('agreeable', 0.6207250952720642),
 ('interesting', 0.6174749732017517)]
```

Then, we use the scrapy-spider model:

```
model =  
Word2Vec.load("D:\RAWorkAgeDiscrim\Wikicorpus\wikiextractor\Spiders\model_spiders")  
  
print(model.wv.syn0.shape)  
model.most_similar("attractive")  
Output:  
[('desirable', 0.6811007857322693),  
 ('advantageous', 0.6603072881698608),  
 ('unattractive', 0.6249780654907227),  
 ('appealing', 0.6150048971176147),  
 ('hospitable', 0.5845235586166382),  
 ('palatable', 0.5709764361381531),  
 ('favorable', 0.5578073263168335),  
 ('apt', 0.5564395189285278),  
 ('adaptable', 0.5564374923706055),  
 ('comfortable', 0.5478556156158447)]
```

Both models calculate similar words that are correlated with the stereotype “attractive.” For instance, both have the words “desirable” and “unattractive.” However, the scrapy-spider model finds the words “appealing” and “adaptable,” which appear to be more related to job markets and stereotypes.

6. Trying to Refine Treatment of “Experience” in Job-Ad Language

As we explain in the paper, we examined in greater detail job-ad language related to experience because this language could sometimes mean more of the characteristic, sometimes less, and sometimes be unrelated. Therefore, we created an algorithm that coded the level of experience required in a job ad to determine the impact of this potential blind spot in our model on our results.^{vi}

^{vi} This code is available upon request.

References

Ningrum, Panggih Kusuma, Tatdow Pansombut, and Attachai Ueranantasun. 2020. Text mining of online job advertisements to identify direct discrimination during job hunting process: A case study in Indonesia. *Plos One* 15:e0233746.